Python Automata Library (PAL) Cheat Sheet

Conventions
import PythonAutomataLibrary as pal · Safe mode is default. Call **pal.FastMode()** before constructing any PAL object. Put substantive model kernels under **@pal.njit(cache=True)**. **dims** gives grid size per axis: **(nx,)** for 1D, **(nx, ny)** for 2D, **(nx, ny, nz)** for 3D. **x...** means **x**, **x,y**, or **x,y,z** in 1D/2D/3D. Negative dimensions enable wrapping; out-of-range coordinates wrap on wrapped axes and are skipped on nonwrapped axes.

Create state

pal.NewGrid(dims, dtype) — Create a typed lattice storing one value per site.
pal.NewAgentGrid(dims, numAgentProps=0, isStackable=False) — Create agents with optional properties; use **dims=()** for nonspatial agents.
pal.NewPopGrid(dims, capacity=None) — Create integer counts per site; capacity limits total population.
pal.NewPDEgrid(dims) — Create a continuous field for PDE updates.
pal.NewIList() — Create a mutable integer list for collecting/reusing query results.
pal.NewMultinomial() — Create a sampler for binomial/multinomial population draws.
Dimensions — 1–3 axes; a negative dimension wraps that axis, e.g. **(-nx, ny)**.

Shared lattice geometry

Properties — **xDim/yDim/zDim**, **nDims**, **wrapX/Y/Z**; a negative dimension enables wrapping on that axis.
ToI(x...) — Convert 1D/2D/3D coordinates into one linear site index.
ItoX/Y/Z(i) — Convert a linear site index back to its axis coordinate.
pal.MooreHood(dim, excludeCenter=False) — Build offsets for all neighboring sites, including diagonals.
pal.VonNeumannHood(dim, excludeCenter=False) — Build axis-aligned offsets, excluding diagonal neighbors.
pal.CircleHood(dim, rad, excludeCenter=False) — Build offsets within a radius; hood constructors return relative offsets, not absolute indices.
grid.Hood(hood, x...) — Apply relative offsets at a coordinate; **excludeCenter=True** omits the center when constructing the hood.

Grid / common indexing

g[i] — Read or write a site by linear index.
g[x...] — Read or write a site using dimensional coordinates.
Slices — Return detached NumPy copies, not live views; editing a slice does not update the grid.
Box(x1,x2,...) — Select a rectangular region with lower and exclusive upper bounds for each axis, in axis order.
Pattern — **g = pal.NewGrid((40,40), float)** · **g[10,12] = 1.0** · **v = g[10,12]**; supported dtypes include bool, fixed-width integers, and float32/float64.

Draw / output

pal.StartPixWindow(...) — Create a pixel buffer and display window; **headless=True** supports noninteractive rendering.
pix[x,y] = RGB — Set a pixel color in the shared pixel buffer.
win.Update() — Refresh the displayed window after drawing.
win.Save(path, block=True) — Save the current image; blocking waits for completion.
win.Close() — Close the window and release display resources.
StartGif(path,delay=100) — Begin recording an animated GIF with the requested frame delay.
AddGifFrame(block=False) — Append the current rendered frame to the GIF.
StopGif() — Finish the GIF; update the window before capturing frames.
pal.StartOpenGLWindow(xDim,yDim,zDim=None,width=800,height=800,...) — Create the renderer and return **(draw, win)**: **draw** collects primitives; **win** handles display, updates, and output. Omit **zDim** for 2D; set it for 3D. **headless=True** supports noninteractive rendering.
draw.Circle(rad,color,x,y,z=None) — Add a circle or sphere-like primitive with the given radius and color.
draw.Box(xLen,color,x,y,z=None,yLen=None,zLen=None) — Add a box centered at a position; specify axis lengths to control its size.
draw.BoxSQ(color,x,y,z=None) — Add a box aligned to a lattice site.
draw.Line(width,color,x1,y1,x2,y2,z1=None,z2=None) — Add a line segment between two 2D/3D points.
draw.Borders(width,color) — Draw grid borders with the requested line width and color.
win.Camera(x,y,z,yaw=None,pitch=None) — Set the camera position and optional yaw/pitch angles for a 3D scene.
win.Background(color) — Set the scene background color.
draw.Clear() / win.Clear() — Clear collected primitives or clear the rendered scene, depending on which object is called.
win.Update() / win.IsOpen() — Render pending drawing commands or check whether the window remains open.
win.Save(path,block=False) — Save the rendered image; **block=True** waits for completion.
win.StartGif(...) / win.AddGifFrame(...) / win.StopGif(...) — Start, append frames to, and finish a GIF recording.
win.Close() — Close the OpenGL window and release its resources.

Lists + randomness

IList methods — **Append(i)** adds an integer; **Clear()** empties the list; **Random()** picks an entry; **Shuffle()** reorders entries; **All()** copies the list; **Iter()** traverses without copying.
pal.Seed(seed) — Seed PAL's shared random stream for reproducible call sequences.
pal.Random() — Draw a uniform random number from PAL's random stream.
pal.RandInt(n) — Draw an integer from 0 through n–1; n must be positive. PAL's stream is separate from NumPy's RNG.
m.Binomial(n,p) — Draw a binomial count with n trials and probability p.
m.Setup(...) / m.Sample(...) — Configure a multinomial sampler, then draw counts from it.

AgentGrid

NewAgentSQ(x...) / MoveSQ(a,x...) — Create or move an agent using lattice-site coordinates; documented linear-site forms are also supported.
NewAgent(x...) / Move(a,x...) — Create or move an agent using continuous coordinates rather than lattice sites.
grid[a,p] — Read or write property p for agent a.
I(a) — Get an agent's linear lattice-site index.
XSQ/YSQ/ZSQ(a) — Get integer lattice coordinates; **X/Y/Z(a)** return continuous positions.
Alive(a) / Dispose(a) — Check whether an agent is still valid, or remove it from the grid.
GetPop() — Count agents currently in the grid.
AgentsAt(x...) / LastAgent(x...) — Find agents at a site or retrieve the last agent there.
AgentsInRadius(...) — Find agents near a position; wrapped displacement respects periodic boundaries.
counts[x...] — Read site occupancy counts without modifying them.

Python Automata Library (PAL) Cheat Sheet

Iteration — **for a in agents.All(): ...** iterates a detached snapshot and is safe for structural mutation such as **Dispose**. **AgentsInRadius(...)** returns nearby agents; wrapped displacements account for periodic boundaries.
DispWrapX/Y/Z(p1,p2) — Compute the shortest displacement along a periodic axis.

PopGrid + PDEgrid

Add(v,...) — Accumulate pending changes without immediately changing current values.
Update() — Apply accumulated changes together as a synchronous update.
Reset() — Clear both current values and pending changes; direct indexed assignment changes current state immediately.
pop.GetPop() — Return total population across all sites.
pop.All() — Return indices of sites with nonzero population, not the population values.
PDE setup — **field.SetTimeSpaceStep(dt, dx[,dy,dz])** sets time and spatial steps before updates. Use spacing values that satisfy the selected scheme's stability requirements.
Diffusion(...) — Diffuse the field using the standard lattice scheme.
DiffusionMask(...) — Diffuse subject to a mask of permitted sites or interfaces.
DiffusionField(...) — Use another field to vary diffusion across space.
DiffusionInterfaces(...) — Specify diffusion behavior at interfaces.
DiffusionADI(...) — Use an alternating-direction implicit diffusion scheme.
DiffusionRadialCircle/Sphere(...) — Use radial diffusion for circular or spherical geometry.
Advection(...) — Move field values with a specified velocity.
AdvectionField(...) — Use a field to define spatially varying advection.
AdvectionInterfaces(...) — Specify advection behavior at interfaces.
More detail: Manual · API Guide · API Reference (Documentation/).